

Informe de reflexión CI/CD

Inventario App · Docker · GitHub Actions · Kubernetes · Blue-Green

Integrantes
José Vanegas · Miguel Vanegas

Repositorio público
github.com/migu145/inventario-app

01 Síntesis del trabajo

La práctica llevó una aplicación Node.js desde pruebas locales hasta dos versiones desplegadas en Minikube. El flujo usa un Dockerfile multi-stage que ejecuta la suite antes de construir producción, dos jobs fail-fast en GitHub Actions, publicación por SHA en GHCR, Rolling Update y una estrategia Blue-Green nativa de Kubernetes. También se implementaron los tres componentes adicionales: Secret, escaneo Trivy y readiness con arranque lento real.

02 Por qué se eligió Blue-Green

Blue-Green encaja mejor que Canary en este laboratorio porque cada respuesta de `/version` identifica versión, color y pod. Mantener dos Deployments independientes permite validar Green completo sin mezclar respuestas con Blue. El corte se realiza cambiando únicamente el selector slot del Service, sin reconstruir imágenes ni reemplazar pods durante la promoción.

La decisión usa solo recursos nativos: `inventario-app-blue`, `inventario-app-green` e `inventario-service`. Frente a Canary, consume más recursos porque duplica dos réplicas, pero ofrece una demostración más determinista y un rollback inmediato: si Green falla, el selector vuelve a `slot=blue`.

03 Qué ocurrió con los datos al recrear el pod

El producto agregado desde la interfaz se escribió en `/app/data/products.json` dentro del contenedor que atendió la petición. Al eliminar ese pod, Kubernetes creó otro a partir de la imagen, pero no restauró el archivo modificado. El producto dejó de aparecer. Con dos réplicas también pueden observarse catálogos distintos, porque cada pod mantiene su propia copia local.

Conclusión de persistencia: un Deployment mantiene procesos, no convierte el sistema de archivos del contenedor en almacenamiento compartido. En producción se necesitaría un PersistentVolume apropiado o, preferiblemente para varias réplicas, una base de datos externa.

04 Métricas DORA propias

Los tiempos de commit provienen de Git y los tiempos de despliegue se registraron inmediatamente después de que kubectl rollout status confirmó cada versión. No se utilizó la hora de publicación de GHCR como sustituto del despliegue.

00:02:55	2 por día	33,33 %
Lead time promedio	Frecuencia de despliegue	Change failure rate

Versión	Commit	Despliegue correcto	Lead time
Release 1	25-jul 16:57:29	25-jul 17:01:00	00:03:31
Release 2	25-jul 17:02:01	25-jul 17:04:20	00:02:19

Cálculos: dos promociones correctas en un día equivalen a 2 despliegues/día. El promedio de 00:03:31 y 00:02:19 es 00:02:55. En el conjunto auditado hubo tres intentos versionados: una etiqueta inexistente falló y las dos imágenes corregidas funcionaron, por lo que $1 / 3 \times 100 = 33,33 \%$. El CFR conserva de forma transparente el fallo histórico y sigue siendo la principal oportunidad de mejora.

05 Problemas reales y resolución

- BuildKit omitía las pruebas. La etapa final no dependía de test. Se corrigió copiando el código desde esa etapa, lo que obliga a ejecutar las seis pruebas.
- ImagePullBackOff. Un manifiesto apuntó a una etiqueta SHA inexistente. Se verificaron las etiquetas publicadas en GHCR y se reemplazó por el SHA Blue válido.
- Arranque lento incompleto. El YAML declaraba STARTUP_DELAY_SECONDS, pero /health no la leía. Ahora devuelve 503 durante la espera y 200 cuando está listo; una prueba controla ambos estados.
- Trivy no descargaba su base. Dos repositorios agotaron el timeout. Se confirmó que la caché estaba vacía, se usó el repositorio público alternativo y el escaneo final reportó cero hallazgos CRITICAL.
- PowerShell alteró el JSON de kubectl patch. Se movieron los parches a archivos JSON y se aplicaron con --patch-file.

06 Reflexión final

La práctica mostró que desplegar no es solo copiar una imagen: requiere conocer qué versión está activa, impedir tráfico prematuro, conservar una ruta de rollback, tratar los secretos fuera de Git y medir el proceso con datos. La evidencia de fallos resultó tan útil como la de éxito, porque permitió corregir el pipeline y señalar como siguiente prioridad la validación automática de etiquetas antes del despliegue.